TripMate AI - Agents, Google ADK & Evaluation (Detailed Reference)

A complete walkthrough of every agent in the TripMate AI travel planner, how it is orchestrated on **Google ADK** (sessions, state, memory), and how each run is **evaluated and scored**.

Scope: worldwide trips (any country). Set `TRAVEL_SCOPE=india` to restore the India-only policy. **Stack:** Google ADK (orchestration/runtime) - Ollama `llama3.2` or Groq free tier (LLM) - MCP tools - Responsible-AI guardrails - automated evaluation - SQLite (preferences, memory, sessions). **Free and local by default** - no paid services.

---

1. System at a glance

TripMate AI is a **multi-agent** workflow: a Supervisor understands the request and dynamically picks which specialist agents to run; the specialists work in parallel; a Final agent composes the plan; then the plan is automatically **scored**. Two control gates (Input Guardrail and Human Review) protect and supervise the flow. Google ADK orchestrates everything and manages the **session**, shared **state**, and long-term **memory**.

```
user request
     |
     v
[ Input Guardrail ] --block--> Final (reason)        control gate
     | pass
     v
[ Supervisor ] --clarify / out-of-scope--> Final     AI agent (routing + extraction)
     |
     v
[ Human Review ] (optional approve/cancel)           control gate (HITL)
     |
     +--> Flight  Hotel  Weather  Budget  Itinerary   (selected subset, in parallel)
     |
     v
[ Final Response ] -> plan                            AI agent (composition)
     |
     v
[ Evaluation ] -> score (0-100 + grade)              automated quality scoring

State, session and memory are held by Google ADK (SessionService) + SQLite stores.
Every agent output passes through the Responsible-AI guard (PII redaction, safety, disclaimer).
```

Agent count: 7 AI agents (Supervisor + 5 specialists + Final) and 2 control gates (Input Guardrail, Human Review), followed by an automated Evaluation step.

---

2. Each agent in detail

For every agent: its **purpose**, the **state it reads**, the **state it writes**, the **tools/LLM** it uses, and its **file**. All agent outputs are post-processed by the Responsible-AI output guard (`travel_agent/responsible_ai.py`): PII redaction, a safety scan, and a grounding disclaimer, with one transparency event appended to `rai_log`.

2.1 Input Guardrail (control gate)

- **File:** `travel_agent/agents/guardrail.py`
- **Purpose:** validate every request before any work begins.

- **Checks:** relevance, safety, prompt-injection / jailbreak detection, validity. Rule-based plus **Llama Guard** when available (free), falling back to rules.
- **Reads:** `user_query`, `trip` (follow-up detection). **Writes:** `guardrail_passed`, `guardrail_reason`, `rai_log`, `llm_calls`; if blocked, `final_response` with the reason.

2.2 Supervisor Agent (the "brain")

- **File:** `travel_agent/agents/supervisor.py`
- **Purpose:** understand the request, extract structured **trip constraints**, enforce the configured **scope** (worldwide by default), and **dynamically select** which specialists to run.
- **Reads:** `user_query`, previous `trip`, saved `preferences`, recalled `recalled_memory`.
- **Writes:** `trip` (origin/destination/IATA/country/dates/days/travellers/budget/interests), `selected_agents`, `supervisor_reasoning`, `preferences`, `messages`, `llm_calls`; clarification or out-of-scope `final_response` when needed.
- **Tools / LLM:** two structured LLM calls (`TripDetails` extraction, `SupervisorDecision` selection) with a rule-based fallback, plus worldwide airport/scope lookups (`travel_agent/india.py::resolve_airport`, `out_of_scope`).

2.3 Human Review (control gate, HITL)

- **File:** `travel_agent/agents/human_review.py`
- **Purpose:** optionally pause so a person can review/approve the plan before specialists run.
- **Mechanism:** the orchestrator emits `review_payload(state)` as an interrupt; `apply_decision()` applies approve / cancel / note. No LLM.

2.4 Flight Agent (specialist)

- **File:** `travel_agent/agents/flight_agent.py`
- **Purpose:** best flights and ground transport with booking tips, fares in the **route's local currency** (plus an approximate USD equivalent).
- **Reads:** `trip`. **Writes:** `flight_results`, `llm_calls`.
- **Tools (MCP):** AviationStack schedules + airport info + Tavily web search; LLM summary. Skips if no origin.

2.5 Hotel Agent (specialist)

- **File:** `travel_agent/agents/hotel_agent.py`
- **Purpose:** compare hotels across budget levels; recommend an area (prices in local currency + USD).
- **Reads:** `trip`. **Writes:** `hotel_results`, `llm_calls`.
- **Tools (MCP):** Google Places + Tavily search; LLM summary.

2.6 Weather Agent (specialist)

- **File:** `travel_agent/agents/weather_agent.py`
- **Purpose:** weather outlook and a packing list for the destination and season - **worldwide**.
- **Reads:** `trip` (destination, start_date, num_days, destination_country). **Writes:** `weather_info`.
- **Tools (MCP):** Custom Weather MCP > **Open-Meteo** (free, no key, global geocoding + forecast).

2.7 Budget Agent (specialist)

- **File:** `travel_agent/agents/budget_agent.py`
- **Purpose:** an itemised cost estimate in the destination's **local currency** (plus an approx USD total) and money-saving tips.
- **Reads:** `trip`, `flight_results`, `hotel_results`. **Writes:** `budget_analysis`. LLM only.

2.8 Itinerary Agent (specialist)

- **File:** `travel_agent/agents/itinerary_agent.py`
- **Purpose:** a realistic day-by-day plan grouping nearby sights, respecting interests, local opening hours and the destination's season/climate.
- **Reads:** `trip`, `weather_info`. **Writes:** `itinerary`.
- **Tools (MCP):** Google Places attractions + Tavily search; LLM.

2.9 Final Response Agent (composition)

- **File:** `travel_agent/agents/final_agent.py`
- **Purpose:** combine the selected reports into one polished plan (overview, flights, stay, weather, itinerary, budget, practical tips incl. visa/entry where relevant).
- **Reads:** all report keys, `trip`. **Writes:** `final_response`, `messages`, `llm_calls`.
- **Tools / LLM:** Tavily tips + LLM composition; plus a final **Llama Guard** safety pass and a disclaimer.

Shared state (`TravelState`) - keys agents exchange

`user_query`, `trip`, `preferences`, `recalled_memory`, `selected_agents`, `supervisor_reasoning`, `guardrail_passed`, `guardrail_reason`, `hitl_feedback`, `flight_results`, `hotel_results`, `weather_info`, `budget_analysis`, `itinerary`, `final_response`, `eval_report`, `llm_calls`, `rai_log`, `messages`.

---

3. Google ADK - orchestration, session, state & memory

Google ADK (`google-adk`, Apache-2.0, free, local) is the runtime; it replaced LangGraph in the app. The orchestrator is `travel_agent/orchestrator.py`, exposing the `TravelPlanner` class the UI/CLI use.

3.1 What ADK replaced

Previously (LangGraph)	Now (Google ADK)
`StateGraph` + nodes/edges	ADK orchestration in `TravelPlanner` over an ADK `SessionService`
Checkpointer for state	ADK `DatabaseSessionService` (durable SQLite)
`add_messages` / reducers	A merge function in the orchestrator (`_apply`)
`interrupt()` for HITL	Session-based pause/resume driven by the orchestrator
LangGraph store (preferences)	SQLite `PreferenceStore`

3.2 Session management

- Each conversation (`thread_id`) is an **ADK session** keyed by `(app_name, user_id, session_id)`.
- Default backend is **`DatabaseSessionService` over SQLite** (`adk_sessions.db`, async `aiosqlite`), so sessions and history **survive restarts** (`backend = "adk-sqlite"`); in-memory is the fallback.
- A background asyncio loop (`_Loop`) lets the synchronous app drive ADK's async session API.
- A persistent **thread>user index** lets a reopened conversation resolve and reload after a restart.

3.3 State management

- The shared `TravelState` lives in the ADK session's `state`.
- Each step persists its update as an ADK **`Event`** with a **`state_delta`** (via `append_event`) - an auditable event log.
- Reducers in `_apply()`: **sum** `llm_calls`, **concat** `rai_log`, **append** `messages`, overwrite the rest. Parallel specialist updates merge safely (distinct keys).

3.4 Memory (three layers, all free/local)

1. **Preference memory** - `PreferenceStore` (SQLite `preferences.db`): lasting user preferences, saved by the Supervisor and reused on every future request.
2. **Long-term trip memory** - `MemoryStore` (SQLite `memory.db`): a one-line summary saved after each completed plan; the next request **recalls** the user's recent trips into `recalled_memory` (shown to the agents via `trip_summary`). Persists across restarts and across different conversations.
3. **ADK MemoryService** - completed sessions are ingested into ADK's `InMemoryMemoryService`.

3.5 Orchestration API (`TravelPlanner`)

- **`stream(query, thread, user, hitl)`** - ensure session, recall memory, reset per-run keys, then Guardrail

  > Supervisor > (clarify/scope > Final) > Human Review (pause if `hitl`) > selected specialists **in parallel** > Final > **Evaluation** > save memory. Yields `(node, update)`; a pause surfaces as `("__interrupt__", payload)`.

- **`resume(decision, …)`** - continues a paused run (approve/cancel/note).
- **`plan(...)`**, **`get_values`** / **`history`** / **`preferences`** / **`clear_preferences`** - reads over the session and stores.

---

4. Evaluation - scoring each run

After a real plan completes, an automated **Evaluation** step scores it and the score is shown in the UI. Implemented in `travel_agent/evaluation.py`; the result is stored in state as `eval_report` and streamed as an `evaluation` node.

4.1 Heuristic metrics (deterministic, no model call)

Metric	What it checks
Completeness	Every selected specialist produced output and the final plan has the expected sections
Safety	Guardrail passed, 0 PII leaks, no unsafe flags (reads `rai_log`)
Grounding	Plan shows a currency, itinerary covers `num_days`, weather present when selected
Personalization	Saved preferences / recalled memory are reflected in the plan (N/A > full marks)
Efficiency	Banded from `llm_calls` and run latency

4.2 LLM-as-judge (optional, 1 call)

A `JudgeScore` rubric scores **relevance / coherence / helpfulness** (0-10 each) plus a one-sentence rationale, using the project's own free model (`get_structured_llm`). Skipped in demo mode or when `ENABLE_LLM_JUDGE=false`.

4.3 Overall score & where it appears

Heuristics + judge are combined into an **overall 0-100 with an A-F grade**. In Streamlit a **" Evaluation" tab** shows the overall score, a per-metric breakdown and the judge's rationale; a **score badge** appears on the Final-plan tab. (Example live result: overall 78 / grade C, with the judge noting a relevance gap - the evaluator genuinely surfaces weak plans.)

4.4 Offline suite

`evaluate_suite.py` scores a set of sample trips (India + international) and prints a per-case table + aggregate (non-zero exit if the average is weak - useful in CI). Run `python evaluate_suite.py` (demo) or `--live`. ADK's own `adk eval` framework is Runner/Vertex-centric, so the app uses this self-contained, free suite instead.

5. Technology summary

Area	Choice
Orchestration & runtime	**Google ADK** (`google-adk`)
Sessions (durable)	ADK `DatabaseSessionService` + SQLite (`aiosqlite`, `greenlet`)
State	`TravelState` in ADK session, reducer merge in the orchestrator
Memory	SQLite `PreferenceStore` + `MemoryStore` + ADK `InMemoryMemoryService`
LLM	Ollama `llama3.2` (free/local) or Groq free tier
Safety model	Llama Guard (free) with rule-based fallback
Tools	MCP servers: Open-Meteo (weather), AviationStack (flights), Tavily/Google (search/places)
Responsible AI	PII redaction, safety scan, injection defense, disclaimers (every agent)
Evaluation	Heuristic metrics + LLM-as-judge; UI tab + offline suite
Scope	Worldwide (default) or India-only (`TRAVEL_SCOPE=india`)
UI	Streamlit web app (login, progress, result tabs, RAI panel, Evaluation tab, memory view)

---

6. How to run

```
.venv\Scripts\python -m streamlit run streamlit_app.py
# open http://localhost:8502
```

Key settings (all optional, in `.env`): `TRAVEL_SCOPE` (global/india), `ENABLE_LLM_JUDGE` (true/false), `ENABLE_RAI`, `LLM_PROVIDER` (auto/ollama/groq), `GROQ_API_KEY` (free faster LLM), `ADK_SQLITE_SESSIONS` (session DB URL). Offline scoring: `python evaluate_suite.py`.

TripMate AI - a worldwide, multi-agent, Responsible-AI travel planner orchestrated on Google ADK, with automated evaluation. Free and local.